

Session 2 : Transformations et Espaces de Coordonnées

💡 Objectifs de la session

Comprendre le voyage d'un vertex à travers les espaces de coordonnées, maîtriser les matrices de transformation, et exploiter la hiérarchie de scène (parenting) pour animer des objets complexes.

Le voyage d'un Vertex : Le Pipeline de Transformation

Un vertex possède plusieurs positions

Un vertex ne possède pas "une" position, mais une série de positions selon le référentiel dans lequel on l'observe. À chaque étape, une matrice transforme les coordonnées d'un espace à l'autre.



Figure 1: Un vertex voyage d'un espace à l'autre jusqu'à ce qu'il devienne un pixel à l'écran.

Espace Local (Object Space)

L'origine est au centre de l'objet $(0, 0, 0)$. C'est l'espace dans lequel l'artiste modélise le mesh. Les coordonnées sont relatives à l'objet lui-même, indépendamment de sa position dans la scène.

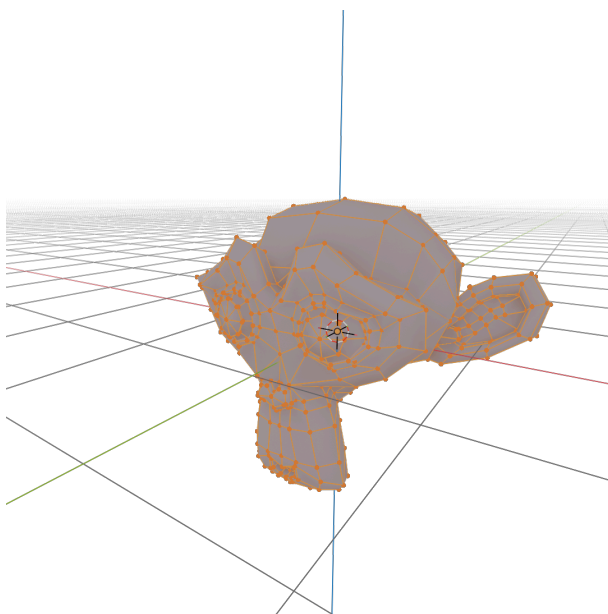


Figure 2: Espace Local : l'origine est au centre de l'objet.

Espace Monde (World Space)

La position de l'objet dans la scène globale. Tous les objets y sont exprimés dans un repère commun.

Transformation : Matrice Monde (Model Matrix) — combine la position, rotation et échelle de l'objet pour passer de l'espace local à l'espace monde.

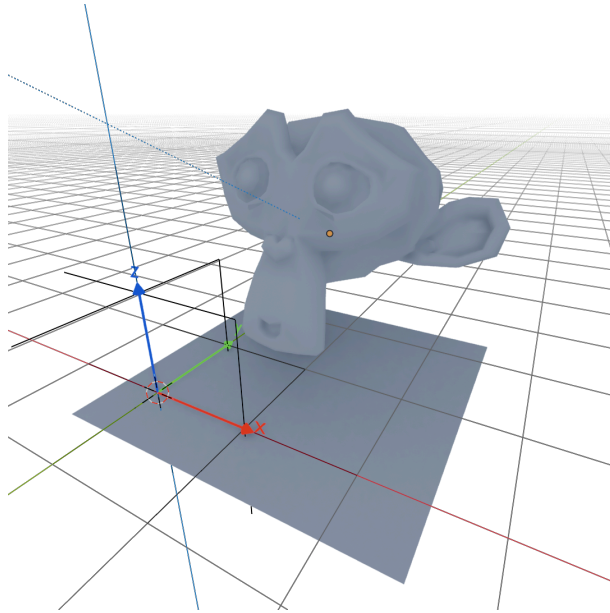


Figure 3: Espace Monde : tous les objets partagent un repère commun.

Espace Caméra (View Space)

Le monde vu depuis l'œil du joueur. La caméra devient l'origine $(0, 0, 0)$, et les axes sont alignés sur son orientation.

Transformation : Matrice Vue (View Matrix) — transforme tout le monde pour qu'il soit vu depuis la caméra.

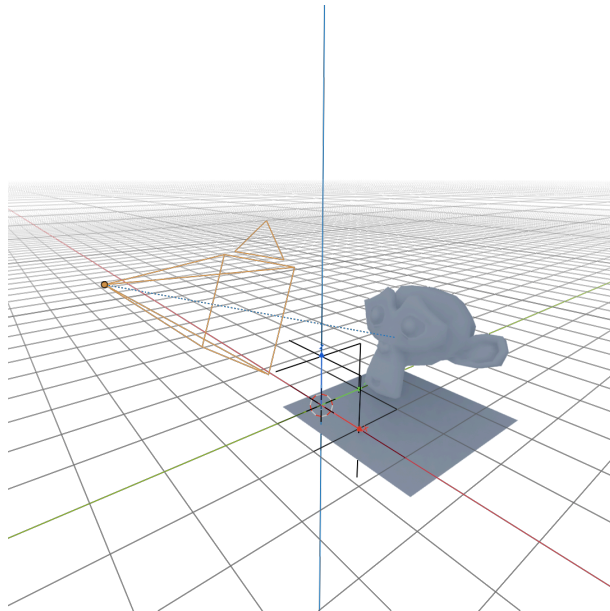


Figure 4: Espace Caméra : la caméra devient l'origine, le monde est vu depuis son point de vue.

Espace Projection (Clip Space)

Conversion du volume 3D en un cube plat $[-1, 1]$. C'est ici qu'on définit la perspective : plus un objet est loin, plus il apparaît petit.

Transformation : Matrice de Projection — projette les coordonnées 3D en coordonnées 2D normalisées (NDC).

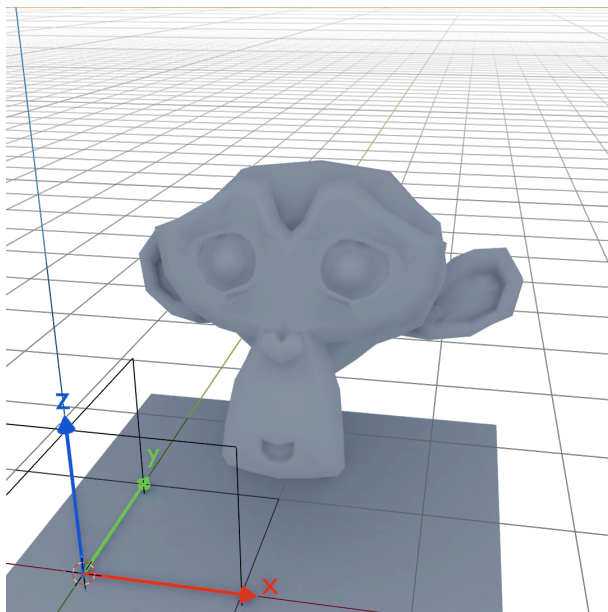


Figure 5: Espace Projection : le volume 3D est projeté dans un cube normalisé $[-1, 1]$.

Comprendre les Matrices : L'approche "Calculatrice"

La matrice comme machine

Pour commencer, oubliez les modèles 3D et les caméras. Ouvrez simplement la console de votre **Playground** Babylon.js. Nous allons observer la matrice pour ce qu'elle est réellement : une simple "machine" mathématique qui prend un point en entrée, le transforme, et recrache un nouveau point.

La Matrice Identité : La machine au repos

Définition

La Matrice Identité est une grille 4x4 avec une diagonale de 1 et des 0 partout ailleurs. En algèbre matricielle, c'est l'équivalent du chiffre 1 dans une multiplication classique ($5 \times 1 = 5$).

Micro-exercice 1 : Prouvez que cette matrice ne fait rien. **Consigne** : Créez un point (un `Vector3`) à la position (3, 4, 5). Créez ensuite une Matrice Identité. Faites passer votre point dans cette matrice à l'aide de la fonction `TransformCoordinates` et affichez le résultat dans la console.

```
let p = new BABYLON.Vector3(3, 4, 5);
let mIdentite = BABYLON.Matrix.Identity();

// On transforme le point avec la matrice
let resultat = BABYLON.Vector3.TransformCoordinates(p, mIdentite);
console.log("Résultat Identité :", resultat); // Affiche {X:3, Y:4, Z:5}
```

La Matrice de Scale : Le multiplicateur

Principe

Si vous remplacez les 1 de la diagonale par d'autres chiffres, la machine devient un multiplicateur. Le premier chiffre touche l'axe X, le deuxième le Y, et le troisième le Z.

Micro-exercice 2 : Aplatis un point. **Consigne :** Prenez le point (2, 2, 2). Fabriquez une matrice de **Scale** qui double sa taille en X et Y, mais l'écrase complètement à 0 sur l'axe Z. Affichez le résultat.

```
let p2 = new BABYLON.Vector3(2, 2, 2);
let mScale = BABYLON.Matrix.Scaling(2, 2, 0); // Z est multiplié par 0 !

let resultatScale = BABYLON.Vector3.TransformCoordinates(p2, mScale);
console.log("Résultat Scale :", resultatScale); // Affiche {X:4, Y:4, Z:0}
```

Micro-exercice 3 : Le téléporteur. **Consigne :** Vous êtes à l'origine (0, 0, 0). Utilisez une matrice de translation pour vous téléporter aux coordonnées (10, 5, -2).

```
let p3 = new BABYLON.Vector3(0, 0, 0);
let mTrans = BABYLON.Matrix.Translation(10, 5, -2);

let resultatTrans = BABYLON.Vector3.TransformCoordinates(p3, mTrans);
console.log("Résultat Translation :", resultatTrans); // Affiche {X:10, Y:5, Z:-2}
```

Le laboratoire de test : L'ordre des opérations

Combiner les transformations

Maintenant que vous possédez les briques de base, nous allons combiner ces opérations. C'est ici que le pouvoir de la matrice prend tout son sens : on peut fusionner des dizaines de transformations en une seule "Super Matrice".

Pour fusionner deux matrices, il faut les multiplier. Mais attention : en mathématiques, l'ordre dans lequel vous multipliez vos matrices change complètement le résultat. $A \times B \neq B \times A$.

Micro-exercice 4 : Le piège de l'ordre. **Consigne :** Vous avez un point de départ à X = 1. Vous disposez d'une matrice Scale x2 et d'une matrice Translation +3.

- **Cas A :** Fusionnez les matrices pour appliquer le **Scale** d'abord, puis la **Translation**. Quel est le résultat ?
- **Cas B :** Fusionnez les matrices pour appliquer la **Translation** d'abord, puis le **Scale**. Quel est le résultat ?

```
let point = new BABYLON.Vector3(1, 0, 0);
let S = BABYLON.Matrix.Scaling(2, 1, 1);
let T = BABYLON.Matrix.Translation(3, 0, 0);

// Cas A : T * S (On scale d'abord, puis on translate)
// En math : (1 * 2) + 3 = 5
let matriceA = S.multiply(T);
let resA = BABYLON.Vector3.TransformCoordinates(point, matriceA);
console.log("Cas A :", resA.x); // Affiche 5

// Cas B : S * T (On translate d'abord, puis on scale)
// En math : (1 + 3) * 2 = 8
let matriceB = T.multiply(S);
```

```
let resB = BABYLON.Vector3.TransformCoordinates(point, matriceB);
console.log("Cas B :", resB.x); // Affiche 8
```

🗨 Observation clé

Bouger puis grossir ne donne pas la même position finale que grossir puis bouger ! C'est ce qui crée des comportements étranges (les "orbites") quand vous animez des objets complexes.

Anatomie des matrices de transformation

Matrice de Translation

Déplace un point. La 4ème colonne contient les valeurs de déplacement (t_x, t_y, t_z) .

$$T = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Matrice de Scale (Échelle)

Multiplie les coordonnées par un facteur. La diagonale contient (s_x, s_y, s_z) .

$$S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Matrices de Rotation

Rotation autour de chaque axe. Les axes perpendiculaires sont affectés par $\cos \theta$ et $\sin \theta$.

Rotation X

$$R_x = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation Y

$$R_y = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation Z

$$R_z = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

💡 Observation

La matrice de **Translation** utilise la 4ème colonne (d'où l'importance de $w = 1$), tandis que les matrices de **Rotation** et **Scale** utilisent la zone 3x3 supérieure gauche. C'est cette séparation qui rend les coordonnées homogènes indispensables.

La Matrice de Translation et l'astuce de la coordonnée W

Pourquoi 4x4 ?

Vous vous demandez sûrement pourquoi nous utilisons des matrices 4x4 pour un espace en 3D (X, Y, Z) ? Le problème est que multiplier la diagonale ne permet pas **d'ajouter** (donc de

déplacer) un point. La solution se trouve dans la 4^{ème} colonne de la matrice. Pour que le calcul active cette colonne de déplacement, votre point 3D possède en réalité une 4^{ème} coordonnée cachée : $w = 1$. L'opération mathématique finale fera donc : $\text{NouveauX} = (X \times 1) + (\text{TranslationX} \times 1)$.

Versions 2D des matrices de transformation

Les mêmes transformations existent en 2D (X, Y) avec des matrices 3x3. On retrouve la même structure : la translation dans la 3^{ème} colonne, le scale sur la diagonale, et la rotation dans la zone 2x2.

Translation 2D	Scale 2D	Rotation 2D	Shear X	Shear Y
$\begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & k_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 \\ k_y & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Atelier Desmos : Visualiser les matrices en 2D

Outil

Ouvrez Desmos, calculatrice matricielle : <https://www.desmos.com/matrix>. Desmos permet de définir des matrices et de les multiplier directement. C'est l'outil idéal pour visualiser l'effet d'une matrice sur une forme géométrique avant de passer au 3D.

Étape 1 : Définir un point et la matrice Identité. **Consigne :** Dans Desmos, créez une matrice $M = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ et un point $P = (2, 3)$. Multipliez $M \times P$ pour obtenir un nouveau point P' .

- Entrez : $M = \begin{bmatrix} 1, & 0 \\ 0, & 1 \end{bmatrix}$
- Entrez : $P = (2, 3)$
- Entrez : $P' = M * P$

Résultat attendu : $P' = (2, 3)$. La matrice Identité ne change rien — le point reste à la même position.

Étape 2 : Scale (Étirer la forme). **Consigne :** Remplacez la matrice M par une matrice de scale qui double la taille en X et réduit de moitié en Y.

- Entrez : $M = \begin{bmatrix} 2, & 0 \\ 0, & 0.5 \end{bmatrix}$
- Observez P' : le point s'est déplacé de $(2, 3)$ à $(4, 1.5)$.

Expérience : Créez un polygone (un carré) avec plusieurs points et appliquez la matrice à chacun. Vous verrez la forme s'étirer et s'aplatir en temps réel quand vous modifiez les valeurs de M .

Étape 3 : Rotation (Faire tourner la forme). **Consigne :** Utilisez la matrice de rotation 2D avec un angle θ que vous pouvez animer.

- Entrez : $M = \begin{bmatrix} \cos(\theta), & -\sin(\theta) \\ \sin(\theta), & \cos(\theta) \end{bmatrix}$
- Ajoutez un curseur pour θ (cliquez sur θ puis activez le curseur).
- Créez un carré avec 4 points et appliquez M à chacun.

Résultat : En faisant varier θ de 0 à 2π , la forme tourne autour de l'origine. C'est exactement ce que fait le GPU avec vos modèles 3D, mais avec une matrice 4x4 et un axe supplémentaire.

Étape 4 : Combiner Translation + Rotation (Le piège !). **Consigne** : En 2D, la translation nécessite une matrice 3x3 avec coordonnées homogènes. Définissez une matrice de translation et une matrice de rotation, puis testez les deux ordres de multiplication.

- **Translation** : $T = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}$
- **Rotation** : $R = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$
- **Cas A (Rotation puis Translation)** : $M_A = T * R$
- **Cas B (Translation puis Rotation)** : $M_B = R * T$

Observation : Animez θ et comparez les deux trajectoires. Dans le Cas A, la forme tourne sur elle-même puis est déplacée. Dans le Cas B, la forme orbite autour de l'origine. C'est la démonstration visuelle que $A \times B \neq B \times A$.

Ce que vous venez de comprendre

Desmos vous a permis de voir, en 2D, exactement ce que le GPU fait en 3D avec des matrices 4x4. Chaque point (vertex) de votre modèle est multiplié par une matrice pour déterminer sa position finale à l'écran. L'ordre des multiplications détermine le comportement de l'animation.

Le lien avec la géométrie : La boucle est bouclée

La matrice applique la même règle à tout l'univers

Rappelez-vous : un modèle 3D (Mesh) n'est qu'un immense tableau géant rempli de Vector3. La matrice est la formule magique que l'on va appliquer dans une boucle for sur **absolument tous** ces points.

Micro-exercice 5 : Incarnez la carte graphique (GPU) ! **Consigne** : Voici le code du carré plat de la Session 1. Avant d'envoyer ces positions pour créer la face 3D, vous allez faire le travail du Vertex Shader. Créez une matrice de Scale(2, 0.5, 1). Faites passer chaque point de votre tableau (grâce à une boucle) dans cette matrice pour déformer la géométrie à la main.

```
// Les coordonnées de base de notre carré
let positions = [
    new BABYLON.Vector3(-1, -1, 0),
    new BABYLON.Vector3(1, -1, 0),
    new BABYLON.Vector3(1, 1, 0),
    new BABYLON.Vector3(-1, 1, 0)
];

// Notre matrice de transformation (Super Matrice)
let mScale = BABYLON.Matrix.Scaling(2, 0.5, 1);

// Vous faites ici ce que le Vertex Shader fera des millions de fois par seconde :
for(let i = 0; i < positions.length; i++) {
    positions[i] = BABYLON.Vector3.TransformCoordinates(positions[i], mScale);
}

// Vérifions le premier point :
console.log("Ancien bas-gauche (-1,-1,0) est devenu :", positions[0]);
// Affiche {X:-2, Y:-0.5, Z:0}

// À ce stade, vous pouvez extraire ces valeurs (X,Y,Z) et les injecter
// dans le VertexData.positions
```

🗨️ Félicitations

Vous venez de comprendre la base absolue de la programmation graphique. Quand, plus tard dans le cours, vous utiliserez `monObjet.position.x = 5`, vous saurez que derrière ce code simple, le moteur génère une matrice de translation et l'applique à des milliers de sommets exactement comme vous venez de le faire !

GPU

Graphics Processing Unit

10496 Cores

Lent - Tâches simples

CPU

Graphics Processing Unit

24 Cores

Rapide - Tâches complexes

Figure 6: Le GPU peut effectuer un grand nombre de multiplications de matrices.

Hiérarchie de scène : Le Parenting

Le concept du référentiel local

Les objets 3D sont organisés en hiérarchie parent-enfant. Le concept fondamental à retenir est que l'univers d'un enfant, c'est son parent.

L'origine (0, 0, 0) de l'enfant n'est plus le centre du monde, mais le centre de son parent. Ses axes (X, Y, Z) s'alignent sur ceux de son parent. C'est ce qu'on appelle le **Scene Graph**.

🗨️ Pourquoi ne peut-on pas juste additionner les positions ?

Si un parent est en $X = 10$ et que l'enfant est en $X = 2$ par rapport au parent, on pourrait penser que l'enfant est en $X = 12$ dans le monde.

C'est faux ! Que se passe-t-il si le parent a tourné de 90 degrés ? L'axe X de l'enfant pointe désormais vers l'axe Z du monde. C'est pour cela que la simple addition ne fonctionne pas.

Le GPU doit obligatoirement utiliser la multiplication de matrices pour calculer la vraie position finale :

$$\text{MatriceMonde}_{\text{Enfant}} = \text{MatriceMonde}_{\text{Parent}} \times \text{MatriceLocale}_{\text{Enfant}}$$

Atelier : Le Train et le Passager

Les propriétés de base dans Babylon.js

- `.position` : Toujours **locale** par rapport au parent.
- `.rotation` (ou `.rotationQuaternion`) : Orientation **locale**.
- `.scaling` : Échelle **locale**.

💡 L'objectif de l'atelier

Nous allons créer un “Train” (le parent) et un “Passager” (l’enfant). Nous allons observer ce qui se passe dans la console pour comprendre la différence entre la position que vous tapez dans le code (Locale) et la position réelle calculée par le GPU (Monde).

Micro-exercice 6 : Embarquement immédiat. **Consigne :** Créez une boîte rectangulaire (train) et placez-la à $X = 10$. Créez une petite sphère (passager). Appliquez la propriété `.parent` pour lier le passager au train. Placez le passager à la position locale $X = 2$. Observez les coordonnées.

```
// 1. Le Train (Le repère parent)
const train = BABYLON.MeshBuilder.CreateBox("train", {width: 4, height: 1, depth: 2},
scene);
train.position.x = 10; // Le train est à 10 unités de l'origine du monde

// 2. Le Passager (Le repère enfant)
const passager = BABYLON.MeshBuilder.CreateSphere("passager", {diameter: 0.5},
scene);

// LA MAGIE OPÈRE ICI :
passager.parent = train;

// La position locale du passager (Dans le référentiel du train)
passager.position = new BABYLON.Vector3(2, 1, 0);

// --- OBSERVATION DANS LA CONSOLE ---

// 1. Où est le passager par rapport au train ?
console.log("Position Locale : ", passager.position.x);
// Affiche : 2

// 2. Où est le passager par rapport au monde 3D ?
passager.computeWorldMatrix(true); // On force le moteur à calculer la multiplication
des matrices
let worldPos = passager.getAbsolutePosition();

console.log("Position Monde : ", worldPos.x);
// Affiche : 12 (car le train n'a pas de rotation, l'addition fonctionne
exceptionnellement ici)
```

Micro-exercice 7 : Le crash test (Rotation). **Consigne :** Ajoutez une rotation au train de 90 degrés sur l’axe Y ($\text{Math.PI} / 2$). Affichez à nouveau la position absolue du passager.

```
train.rotation.y = Math.PI / 2; // Tourne le train de 90 degrés

passager.computeWorldMatrix(true);
console.log("Nouvelle Position Monde X : ", passager.getAbsolutePosition().x);
// Affiche : 10 ! (Et non plus 12)

console.log("Nouvelle Position Monde Z : ", passager.getAbsolutePosition().z);
// Affiche : -2 !
```

Explication : La position locale du passager est toujours $X=2$. Mais comme le train a tourné, l’axe X du train correspond maintenant à l’axe Z négatif du monde. La matrice a calculé cette conversion complexe pour vous !

Concepts clés à retenir (Les pièges classiques)

💡 L'héritage du Scale (L'effet Œuf)

Testez ceci : Appliquez `train.scaling.x = 2` pour allonger votre train. Que se passe-t-il ?

Votre passager (la sphère) se transforme en œuf aplati ! Pourquoi ? Parce que la matrice monde du train contient un **Scale x2**. En multipliant cette matrice, le GPU multiplie aussi les sommets de l'enfant par 2 sur cet axe.

Bonne pratique : En production de jeux vidéo, on évite presque toujours d'appliquer un "Scale" à un objet parent, pour éviter de déformer accidentellement tous ses enfants.

Transformation de vecteurs vs points

Rappelez-vous la coordonnée w . Les normales (l'orientation des faces pour la lumière) sont des **vecteurs** ($w = 0$), pas des points ($w = 1$). Elles ne subissent donc **pas** la translation de la matrice — seulement la rotation. C'est pourquoi une face orientée vers le haut pointe toujours vers le haut, même si on déplace l'objet de 1000 kilomètres.

💡 Question de réflexion pour la prochaine session

"Si je déplace ma caméra de 5 unités vers la droite, que doit faire le GPU avec tous les objets du monde pour qu'ils apparaissent correctement à l'écran ?"

Réponse : Il doit tous les déplacer de 5 unités vers la **gauche**. C'est le rôle de la Matrice Vue (View Matrix) — dans le moteur 3D, la caméra est fixe à l'origine, c'est l'univers entier qui bouge en sens inverse autour d'elle pour simuler le mouvement !